

LOCALMARKET

Master Product Specification

Powered by LocalEats

Version	Master Product Specification v1.0
Date	23 September 2026
GitHub baseline	GitHub main commit 31ef614006d3a001abc3daaff9e363af29df771f
Repository	mehey9184-maker/LocalMarket-POS-Pawn-Manager

Purpose: Persistent product memory for future development sessions. Future AI Studio sessions should read this before changing workflows, roles, navigation, or data models.

1. PRODUCT PURPOSE

LocalMarket is a shop operating system for second-hand retail and pawn businesses. It is designed to manage the whole shop while remaining fast enough for a busy cashier counter.

The software must help staff do their existing work faster rather than trying to replace the worker. The product should feel practical, learnable, calm, and psychologically supportive.

The shop owner should experience LocalMarket primarily as their own branded business software. LocalMarket and Powered by LocalEats should remain visible in appropriate platform/support/about areas without overpowering the shop brand.

2. CORE PRODUCT PRINCIPLE

REMOVE WORK, DO NOT CREATE WORK.

Every workflow should ask only for information that is necessary for that specific job. Unknown historical information is a valid state. Never force users to invent a person, price, date, source, or verification just to satisfy a form.

Use progressive disclosure: normal workers see the minimum needed to complete the task; managers see more controls; owners see business-level configuration and reporting; technical platform administration remains hidden from normal shop users.

3. CORE BUSINESS MODEL

LocalMarket has three distinct ways stock enters the business and one main way stock leaves the business.

- 1) Existing Stock: the shop already owns the item. No person record is required.
- 2) Buy From Person: the shop buys an item from a real seller today. Seller history is retained.
- 3) Pawn: the shop takes an item as collateral from a real pawn customer. Customer and loan records are retained.
- 4) Retail Sale: the shop sells stock. A retail buyer normally does not become a persistent customer record.

4. FINAL NAVIGATION MODEL

Primary navigation should be: Home, Sell, Add Stock, Inventory, Customers, Profile.

Add Stock contains the three acquisition paths: Existing Stock, Buy From Person, Pawn.

Compliance, vault, staff management, reports, and advanced settings should appear inside the relevant workspaces or manager/owner areas rather than as a crowded top-level navigation bar.

The global scanner button should remain removed. Scanning is contextual to the workflow where it is useful.

5. HOME DASHBOARD

Recommendation: make Home a daily operating dashboard rather than a dense analytics dashboard.

Owner and manager view: Sales Today, Items Sold Today, Active Pawns, Loans Due Soon/Overdue, Stock Needing Attention, Recent Activity, and quick actions.

Cashier view: greeting, Sell, Add Stock, Find Item, Find Customer, and small recent activity. Hide management-only figures such as profit and acquisition cost unless permitted by role.

Alerts should be clickable and lead directly to the relevant record. Guidance should be quiet and contextual rather than tutorial popups.

6. USER ROLES AND PERMISSIONS

Expose only two shop-managed staff roles in the interface: Cashier and Manager. The technical admin role remains in the database/platform but must be hidden from normal owners and staff.

Owner is the business-account controller and should have owner-level access to shop setup, staff, reports, branding, business settings, and sensitive financial information.

Cashier: sell, search, add stock, buy from sellers, create pawns, use customer/seller lookup, and complete permitted counter operations. No unrestricted historical editing, no sensitive cost/profit visibility, and no unrestricted refunds/voids.

Manager: cashier capabilities plus approvals, refunds/voids, price changes, more reporting, and controlled operational administration.

Owner: full shop management, staff, branding, business profile, reports, and multi-branch capabilities when enabled.

7. OWNER ONBOARDING

Onboarding must use: Basic setup -> start using app -> complete legal/business profile later.

Initial setup should request only the essentials: shop name, logo, phone/contact information, and a basic address/location. The owner can start working immediately.

After entry, show a quiet progress indicator for incomplete business setup. Legal details, VAT, SAPS licence, receipt settings, and other business data can be completed later.

Never require a new owner to know or configure Supabase manually.

8. SHOP IDENTITY / WHITE-LABEL EXPERIENCE

Each shop should be able to configure: business name, logo, tagline, accent color, receipt header/footer, phone, WhatsApp/contact information, address, and other presentation settings.

The selected shop identity should appear throughout the app UI, login experience, receipts, contracts, printable item tags, and customer-facing outputs where appropriate.

LocalMarket branding remains in subtle platform locations such as login/support/about/version/footer areas, with Powered by LocalEats available as the platform attribution.

The database already contains shop_profiles and a metadata JSONB field; major branding settings should become explicit fields when that is cleaner and easier to query.

9. ACCOUNT CREATION AND LOGIN

Owner login: email + password, with optional PIN for sensitive actions.

Staff login: the shop is remembered on the device after setup; staff use username + 6-digit PIN for fast entry. The UI should not force a cashier to type a shop code repeatedly.

Owner creates cashier/manager accounts from inside the app: Profile/Staff -> Add Cashier or Add Manager -> choose name, username, role, and 6-digit PIN -> create.

Cashiers should be able to remain active for up to 12 hours through a device/session policy. Show the session state and require re-authentication when the session expires.

Add Switch User and Lock Terminal controls in the profile/session area. Switching staff should not require signing out the whole terminal.

Forgot PIN: cashier sees a simple message telling them to ask the owner/manager to reset the PIN. The owner performs the reset inside Staff management.

Important backend correction: do not auto-assign new users to the first active shop. New users must be associated with the correct shop explicitly.

10. EXISTING STOCK FLOW

Existing Stock is a first-class acquisition path.

Flow: Add Stock -> Existing Stock -> Item Details -> Price -> Location -> Complete.

No Seller record. No Customer record. No SellerTransaction. No PawnLoan. No invented owner.

Fields: title, category, brand, model, serial/IMEI where applicable, condition, photo(s), retail price, optional cost basis, stock location, optional internal note, optional source note.

Provenance should support Existing Stock / Unknown / Not recorded. Honest unknown values are preferred to fake historical data.

Every item receives an automatic LocalMarket SKU/asset tag such as LM-000184.

11. BUY FROM PERSON FLOW

Flow: Add Stock -> Buy From Person -> Seller Identification -> Item Details -> Valuation -> Deal Review -> Complete.

When a seller enters their ID number, the system should search the current shop records. If a match exists, show the sellers name immediately so staff can click the correct record.

Search should also support seller name, mobile number, and previous transaction/item references where practical.

Existing seller selection should show a small history summary such as transaction count, last item, last transaction date, and past amounts as role permits.

New seller creation uses the actual compliance information required for the business transaction. Do not mix sellers with pawn customers.

12. PAWN FLOW

Flow: Add Stock -> Pawn -> Customer Identification -> Item Details -> Loan Terms -> Pledge Review -> Complete.

For pawn, the customer must be a real persistent Customer record because the relationship continues through active loans, redemption, extensions, and history.

When a returning customer is found, show useful context: active loans, previous loans, outstanding amount, overdue state, and previous pawn activity, subject to permissions.

Compliance remains embedded in this workflow but should not make every screen feel like a police form.

13. RETAIL SALE / POS

Retail buyers normally do not get persistent customer accounts. This avoids unnecessary Supabase rows and counter friction.

Core flow: Search or scan item -> Add to cart -> Payment -> Receipt -> Done.

Customer information is optional for cases such as requested SMS/WhatsApp receipt or warranty/service follow-up.

The POS should be a very fast cashier terminal. Search results should show photo, title, SKU, retail price, location, and availability so the correct item can be selected quickly.

Unique second-hand items generally use quantity 1. Generic stocked merchandise may support quantity greater than 1.

14. ITEM RECORD AND HISTORY

Clicking an item opens a detailed record with sections/tabs: Overview, Photos, History, Sale History, Acquisition, Compliance, Notes.

Cashier view: operational history such as added, price changed, moved, reserved, sold. Hide sensitive acquisition amounts, cost basis, seller payout, and profit.

Manager/owner view: full acquisition details, cost basis, margin/profit, seller, who changed what, timestamps, and audit details according to permission.

Every important state change should append a durable timeline event. Historical financial/compliance records must be preserved rather than overwritten.

15. ITEM PHOTOS

Photos are strongly encouraged and should be integrated directly into item entry.

Preferred minimum set: Front, Back, Serial/IMEI/Identification. Support 3 or more photos and allow additional photos when needed.

Show a visible documentation state such as 1/3 photos - Add photos or 3/3 photos. The UI should prompt users to add a picture without becoming annoying.

Image storage should use Backblaze B2; Supabase stores the image reference/storage key, not the binary image.

16. INVENTORY AND LOCATIONS

Inventory is the center of stock management.

Suggested simple locations: Retail Floor, Back Room, Vault, plus optional shelf/bin text such as Shelf B12. Do not create a complex warehouse system for the normal shop.

Inventory screen should support All Stock, Retail Floor, Secure Vault, Reserved, Flagged, and other relevant states without becoming visually crowded.

Add Stock must be a prominent but clean action from Inventory.

17. SEARCH AND DUPLICATE PROTECTION

Provide one global search that can find customer, seller, SKU, serial/IMEI, pawn ticket, receipt, and item name/description.

Results must clearly identify the result type: Inventory, Seller, Customer, Pawn Loan, Sale/Receipt.

For serialised items, detect duplicate serial/IMEI values within the shop and show a strong warning with the existing item before allowing continued entry when safe to do so.

For seller ID lookups, show the existing sellers name as soon as an exact match is found.

18. PRICE CONTROL

Permanent retail price changes should be controlled by role.

Cashier may use an approved discount or controlled override workflow, but should not freely rewrite the permanent retail price.

Manager/owner can change the actual item price. The item history should record old price, new price, timestamp, and actor.

Cashiers should not see acquisition cost or profit unless a specific role permission allows it.

19. RETURNS, REFUNDS, AND VOIDING

Recommendation: implement returns/refunds because real retail shops need them, but make them controlled operations.

Flow: find receipt -> select item(s) -> reason -> confirm -> inventory/cash update -> audit event.

Cashier should require manager approval for refunds/voids. Manager/owner can approve within permissions.

Every refund/void must be recorded in sale history and audit logs; do not delete the original transaction.

20. CASHIER SHIFTS / CASH CONTROL

Implement a lightweight shift system.

Open Shift -> starting cash -> normal sales/cash movements -> Close Shift -> counted cash -> expected cash -> difference -> confirmation.

Show the cashier only what they need. Owner/manager can inspect shift reconciliation, differences, and cashier activity.

Cash differences should be visible as operational alerts, not silently ignored.

21. BULK INVENTORY IMPORT

Bulk import is essential for onboarding shops with existing stock.

Support CSV and XLSX import.

Template fields should include: SKU (optional), Title, Category, Brand, Model, Serial/IMEI, Condition, Cost, Retail Price, Location, Notes.

Allow default values for location, status, and category at the import level, with per-row overrides.

Process: upload -> parse -> validation/duplicate preview -> user review -> import -> summary.

Missing SKUs should be generated automatically. Invalid rows should be explained rather than silently dropped.

22. REPORTS

Keep first release reporting intentionally small.

Essential reports: Sales Report and Items Sold.

Provide Today, Yesterday, This Week, This Month, and Custom Date filters.

Useful fields: total sales, item count, average sale, sales by cashier, and individual transactions.

Advanced profitability/finance reporting can exist later but should not crowd the first-run experience.

23. COMPLIANCE

Compliance should operate behind the correct workflows.

Buy From Person and Pawn must preserve required statutory/compliance records and histories.

Existing Stock should not be forced into a fake seller/customer compliance record merely because it is being entered into the system after onboarding.

Compliance records are role-controlled and accessible from the relevant item/transaction history and manager/owner areas.

24. OFFLINE-FIRST OPERATION

Dexie/IndexedDB remains the local offline-first layer.

If internet is unavailable, permitted staff should still be able to sell, add existing stock, search local inventory, and perform permitted local workflows. Changes should queue for later sync.

When connectivity returns, sync should be entity-specific, idempotent, retryable, and should never claim success unless the remote write actually succeeded.

Supabase remains the shared business backend, while Dexie is the local operational database/cache/queue.

25. SUPABASE / MULTI-SHOP MODEL

One owner account may eventually manage multiple shops/branches, but a single-shop owner should only see My Shop by default.

When a second shop is added, expose a simple Switch Shop control. Do not make branch management visible unless the owner uses more than one branch.

Every shop business record should be associated with the correct shop_id. Cross-shop leakage must be prevented by RLS and application scoping.

Current schema already includes shop_id across core business tables. Strengthen user/shop provisioning so account creation never silently attaches a person to an unrelated shop.

26. BACKEND DATA MODEL

Core tables: shop_profiles, profiles, shop_items, customers, sellers, seller_transactions, pawn_loans, sales, saps_entries, system_logs.

Existing Stock must use acquisition_type = Existing Stock (or equivalent normalized representation) and must not populate seller_id/customer_id/pawn_ticket_id unless genuinely applicable.

Seller is separate from Customer. SellerTransaction is separate from Sale and PawnLoan.

Use stable UUIDs and idempotent sync. Preserve historical records.

RLS must ensure a user can access only records belonging to their shop/branch according to role.

27. SECURITY AND AUTHENTICATION RULES

Never expose Supabase secret/service-role credentials in the browser.

PINs should not remain as plain sensitive values if a secure hashing/verification mechanism can be implemented on the server. Avoid hardcoded default PINs in production.

Sensitive owner/manager actions should require appropriate role checks and may require re-authentication or a manager PIN.

Audit security-sensitive operations such as staff creation, PIN reset, refunds, price changes, profile changes, and critical compliance actions.

28. VISUAL DESIGN SYSTEM

Visual direction: D - Facebook-style breathing room + professional POS speed. Do not copy Facebook branding, graphics, or colors.

Light theme should be the default. Keep terracotta as the brand accent, not the page background.

Use spacious layouts, generous whitespace, comfortable section gaps, restrained border radius, subtle shadows, readable type, and normal title case for human-facing labels.

Semantic colors: App background #F5F6F8, primary surface #FFFFFF, secondary surface #F8F9FA, text #1F2937, secondary text #6B7280, border #E5E7EB, brand #C85A32, brand hover #A94725, brand soft #FDF0EA, success #15803D, warning #B45309, danger #DC2626, info #2563EB.

The current repo has already transitioned to a light theme in commit 31ef614; continue refining consistency rather than reintroducing the old dark terminal aesthetic.

29. UX PSYCHOLOGY / HUMAN FACTORS

The software should feel like a helpful coworker, not a bureaucratic obstacle.

Primary actions should be obvious without training. A trained cashier should know what to press. Quiet contextual guidance is allowed where it helps a new worker, but avoid tutorial walls, repeated popups, or excessive instructions.

Warnings should scale with importance: missing photo can warn and allow continuation; duplicate IMEI should be a strong warning; pawn without a customer must block completion; cash mismatch should require manager action.

Use clear language such as Add Stock, Sell, Buy From Person, Pawn, Search previous sellers, and Add photo. Avoid overly technical or statutory language on normal operational screens.

30. HARDWARE AND SCANNING

Scanning should be contextual. Sell can scan an item/SKU; Add Stock can scan/enter serial/IMEI; Inventory can scan to locate an item.

Global scanner access should remain removed from the main header.

Hardware integrations should fail honestly and should not present fake scan results or fake verification.

31. OWNER EXPERIENCE

Owner dashboard: Sales Today, Items Sold Today, Stock Count, Active Pawns, Loans Due/Overdue, Recent Staff Activity, Alerts.

Owner settings: Business Profile, Branding, Staff, Reports, Shift Oversight, Compliance/Legal setup, and advanced configuration as needed.

Owner should be able to start using the application before completing every legal field. Incomplete setup should be visible but non-blocking unless a specific transaction legally requires the missing information.

32. CASHIER EXPERIENCE

Cashier home: Sell, Add Stock, Find Item, Find Customer, small activity list.

Cashier should not be burdened with shop setup, legal configuration, business rules, deep reporting, sensitive cost data, or technical sync controls.

Cashier should be able to switch user/lock terminal and maintain a fast 12-hour operational session policy.

33. MANAGER EXPERIENCE

Manager has operational oversight and approval power without exposing platform-level administration.

Manager can review/refund/void sales, control permanent price changes, inspect shifts, view operational reports, and handle permitted customer/seller/compliance tasks.

Owner remains the final authority for staff, branding, business configuration, and multi-shop control.

34. TECHNICAL CLEANUP REQUIRED BEFORE PRODUCTION

Inspect and remove or quarantine legacy screens/components that are no longer part of the normal navigation, especially duplicate intake implementations.

Current repository contains both BuyPawn.tsx and IntakeDesk.tsx. App.tsx currently routes buy-pawn to BuyPawn; IntakeDesk should not become an accidental second source of truth.

Current AppContext still exposes several compatibility/stub methods. These should be reconciled with real domain contexts before production so the app does not silently present non-functional actions.

Profile provisioning, shop selection, RLS, sync, and staff account creation are production-critical and should be treated as architecture work rather than UI-only changes.

35. DEVELOPMENT RULES FOR AI STUDIO

Always inspect the existing repository before changing it.

Use the master product specification as the source of truth for product decisions.

Do not rebuild the app from scratch and do not redesign unrelated features.

Preserve working domain logic and improve it in place.

Never fabricate data, verification, sync success, or upload success.

After implementation, run TypeScript/build/lint where configured, test the primary workflows, and report exactly what changed and what remains.

36. PRIMARY ACCEPTANCE FLOWS

Flow A - New shop: owner creates account -> enters basic shop profile/logo -> enters dashboard -> can add existing stock immediately -> completes legal profile later.

Flow B - Existing stock: Add Stock -> Existing Stock -> item -> photo reminder -> price -> location -> SKU generated -> saved.

Flow C - Seller buy: Add Stock -> Buy From Person -> enter seller ID -> existing seller name appears if found -> select -> item -> photo(s) -> valuation -> deal -> seller transaction + compliance record -> inventory.

Flow D - Pawn: Add Stock -> Pawn -> enter customer ID -> existing customer appears if found -> select/create -> item -> photo(s) -> loan terms -> pledge -> customer + loan + vault/compliance records.

Flow E - Retail sale: Sell -> search/scan item -> cart -> payment -> receipt -> inventory marked sold -> sale recorded. No customer account required by default.

Flow F - Staff: Owner -> Staff -> Add Cashier -> username + 6-digit PIN -> cashier logs into remembered shop -> 12-hour session -> switch/lock user.

Flow G - Bulk import: Owner/manager -> Inventory -> Import Stock -> XLSX/CSV -> preview/validate -> fix/review -> import -> generated SKUs -> summary.

37. NON-NEGOTIABLE RULES

Never force a person record when it is not part of the business event.

Never treat a retail buyer as a persistent customer by default.

Never treat an outright seller as a pawn customer.

Never treat Existing Stock as a SellerTransaction.

Never fabricate missing history.

Never expose sensitive cost/profit data to unauthorized roles.

Never make normal staff configure Supabase.

Never make the owner configure platform internals just to use the app.

Never claim synchronization, image upload, verification, or compliance actions succeeded when they did not.

Always optimize for speed, clarity, and low cognitive load.

38. FUTURE-READY FEATURES (NOT FIRST PRIORITY)

Advanced profit/margin analytics, richer branch reporting, automated supplier management, deeper hardware integrations, customer communications, expanded document/contract workflows, and more advanced warehouse functions can be added later.

These must not be allowed to clutter the core counter workflow.

39. SOURCE / CURRENT REPOSITORY NOTES

Current repository baseline inspected on 23 September 2026.

Latest observed commit: 31ef614006d3a001abc3daaff9e363af29df771f - feat(ui): transition to light theme and update schema.

The repository currently uses React + TypeScript, Dexie/IndexedDB for local storage, Supabase services, role profiles, seller/customer/pawn/sales/SAPS domains, and a light theme.

The repository currently contains shop-aware database tables and role definitions, but account provisioning and some compatibility/stub paths require cleanup before production.

40. HOW THIS DOCUMENT SHOULD BE USED

This is the persistent product memory for LocalMarket. Future AI Studio sessions should read it before making product or workflow changes.

When requirements change, update this document first or as part of the same change. Do not silently invent a new flow in code that contradicts this specification.

When the chat history is unavailable, the repository plus this document should be enough to reconstruct the product intent, core workflows, roles, UX principles, and architecture direction.